

Investigación de proyecto pinguino

Pinguino es un Entorno de Desarrollo Integrado (IDE) de código abierto.

Pinguino es una placa Open Hardware, compatible tanto con microcontroladores Microchip de 8 bits (PIC18F con módulo USB integrado) como con Microchip de 32 bits (PIC32MX ©).

Pinguino utiliza compiladores libres y de código abierto (SDCC, compilador de dispositivos pequeños en C; y gcc-mips-elf, una versión dirigida de gcc para microcontroladores PIC32) disponibles para GNU/Linux, Windows y Mac OS X.

Pinguino es un firmware/bootloader que controla la placa Pinguino.

Pinguino es compatible al 90% con el lenguaje y las bibliotecas Arduino.

Su creador, Jean-Pierre Mandon, buscaba aprovechar la arquitectura de los microcontroladores PIC, que en su momento ofrecían periféricos USB integrados de forma más económica y robusta. Con la evolución hacia los 32 bits, el proyecto adoptó la familia PIC32MX, permitiendo que estudiantes y desarrolladores accedieran a una arquitectura de clase industrial (MIPS) con la misma facilidad de uso que el ecosistema Arduino.

Análisis del Núcleo MIPS32 M4K

A diferencia de los microcontroladores de 8 bits que procesan un byte por ciclo de instrucción, el núcleo MIPS32 M4K en los modelos PIC32 permite procesar palabras de 32 bits. Esto no solo incrementa la velocidad de procesamiento, sino que permite un manejo de memoria mucho más complejo.

El núcleo utiliza un pipeline de 5 etapas, lo que significa que mientras una instrucción se está ejecutando, la siguiente ya se está decodificando y una tercera está siendo buscada en la memoria. Además, incluye un bloque de multiplicación y división por hardware que es vital para algoritmos de control PID, procesamiento de señales digitales (DSP) o cálculos de punto flotante que suelen ser lentos en arquitecturas menores.

Gestión de Memoria y Direccionamiento Virtual

Uno de los puntos más avanzados al diseñar para un Pinguino 32 es entender que el PIC32 utiliza direccionamiento virtual. La memoria se divide en segmentos conocidos como KSEG0 y KSEG1.

El segmento KSEG0 es "cacheable", lo que acelera la ejecución del código al usar la memoria caché interna. Por otro lado, el KSEG1 no es "cacheable" y es donde se mapean los registros de los periféricos y la memoria de programa cuando se requiere un acceso directo y determinista. Al programar en el IDE de Pinguino, estas complejidades se manejan mediante el script de enlazado (linker script), pero para un diseño de hardware propio, es vital saber que el bootloader debe residir en la "Boot Flash Memory" (un bloque de memoria física separado de la memoria principal) para garantizar que el sistema siempre pueda recuperarse o reprogramarse por USB.

El Motor USB Nativo y el Protocolo HID

La característica distintiva de Pinguino es su capacidad de ser programado directamente por USB. El microcontrolador actúa como un dispositivo de clase HID (Human Interface Device). Esto tiene una ventaja técnica enorme: no requiere la instalación de controladores específicos en el sistema operativo del host (Windows, Linux o macOS), ya que estos sistemas reconocen nativamente dispositivos HID como teclados o ratones.

En tu diseño, el motor USB del PIC32 requiere un transceptor interno que se estabiliza con un regulador de 3.3V dedicado. Aunque el chip se alimente externamente, el pin VUSB debe tener un capacitor de filtrado para asegurar que la comunicación de datos no se interrumpa por ruido eléctrico o caídas de tensión durante la enumeración del dispositivo en el bus.

Consideraciones Críticas para el Ruteado de la PCB

Al trabajar con frecuencias que pueden alcanzar los 80 MHz, el diseño físico de la placa deja de ser una simple conexión de cables para convertirse en un sistema de alta frecuencia.

Capacitor de VCORE/VCAP: Este es el error más común en diseños nuevos. El regulador interno que alimenta el núcleo a 1.8V es extremadamente sensible. El capacitor de 10 μ F debe ser de baja ESR y estar colocado lo más cerca posible del pin, con una pista de tierra ancha que regrese directamente al pin GND del microcontrolador.

Oscilador de Cuarzo: Las pistas que conectan el cristal a los pines OSC1 y OSC2 deben ser simétricas y lo más cortas posible. Cualquier capacitancia parásita en estas líneas puede mover la frecuencia de resonancia, lo que causaría que el motor USB falle, ya que este requiere una precisión de reloj muy alta para mantener la sincronización de bits.

Desacoplo de Energía: Se recomienda colocar un capacitor de 100nF (cerámico) por cada par de pines VDD/VSS. Estos actúan como reservorios locales de energía para los picos de conmutación de alta velocidad de las compuertas lógicas internas.

Comparación de Rendimiento y Software

El IDE de Pinguino utiliza el compilador GCC-MIPS, que es un estándar de la industria. Esto permite que el código sea altamente optimizado. En términos de rendimiento, un Pinguino 32 puede manejar tareas que un Arduino convencional no podría, como generar señales de video VGA simples, manejar pantallas TFT a altas velocidades de refresco o gestionar múltiples comunicaciones seriales (UART, SPI, I2C) simultáneamente mediante el uso de DMA (Direct Memory

Access), que permite mover datos entre periféricos y memoria sin intervención del CPU.

Interfaz ICSP para el Primer Arranque

Aunque el objetivo final es usar el USB, tu diseño debe prever el puerto ICSP (In-Circuit Serial Programming). Los microcontroladores PIC vienen de fábrica "vacíos". Por lo tanto, la primera vez que ensambles tu placa, necesitarás un programador para cargar el archivo hexadecimal del bootloader de Pinguino. Una vez cargado este pequeño programa, podrás sellar tu diseño y realizar todas las actualizaciones futuras exclusivamente a través del puerto USB.

El IDE de Pinguino es una herramienta de desarrollo integral diseñada para facilitar la programación de microcontroladores PIC, manteniendo una filosofía de software libre y multiplataforma. A diferencia de otros entornos propietarios, este IDE se destaca por su ligereza y su capacidad para gestionar arquitecturas tanto de 8 como de 32 bits desde una interfaz unificada.

Arquitectura y Desarrollo del Software

El IDE está desarrollado principalmente en Python, lo que le permite ejecutarse de manera fluida en Windows, Linux y macOS. Su diseño se basa en la simplicidad, buscando una experiencia de usuario similar a la de Arduino (basada en el entorno Processing), pero adaptada específicamente a las necesidades y registros de la arquitectura de Microchip. Al estar escrito en Python, el entorno es modular, lo que facilita la actualización de bibliotecas y la adición de soporte para nuevos modelos de microcontroladores sin necesidad de reinstalar todo el núcleo del sistema.

El Toolchain de Compilación

El corazón del IDE de Pinguino no es un solo compilador, sino un conjunto de herramientas especializadas (toolchain) que se activan según el chip que selecciones:

Para modelos de 32 bits (PIC32): El IDE utiliza MIPS-elf GCC. Este es un compilador de grado industrial que traduce el código C/C++ a instrucciones optimizadas para el núcleo MIPS32 M4K. Es capaz de realizar optimizaciones avanzadas de código para aprovechar la memoria caché y el pipeline del procesador.

Para modelos de 8 bits (PIC18): Utiliza SDCC (Small Device C Compiler), un compilador de código abierto muy eficiente para arquitecturas de 8 bits, que reemplaza a los compiladores comerciales como el XC8 de Microchip.

Lenguaje y Bibliotecas

El lenguaje de programación de Pinguino es esencialmente C++, pero con una capa de abstracción que simplifica las tareas comunes. Al igual que en otros entornos de prototipado, el código se organiza en dos funciones principales: `setup()` para la configuración inicial y `loop()` para la ejecución continua.

Sin embargo, para los modelos de 32 bits, el IDE incluye bibliotecas específicas que permiten manejar periféricos avanzados de forma sencilla. Esto incluye el acceso a funciones de DMA (Acceso Directo a Memoria) para mover datos sin cargar el CPU, gestión de interrupciones vectorizadas y el uso del RTCC (Reloj de Tiempo Real y Calendario) integrado en los chips PIC32. Las bibliotecas están escritas de forma que el usuario pueda bajar al nivel de los registros de hardware si necesita un rendimiento extremo, o mantenerse en las funciones de alto nivel para un desarrollo rápido.

Sistema de Carga (Uploader)

Una de las mayores ventajas del IDE es su sistema de carga integrado. Utiliza el protocolo HID (Human Interface Device) para comunicarse con el bootloader que reside en el microcontrolador.

Esto significa que cuando presionas el botón de "Subir", el IDE busca un dispositivo USB con un ID específico (Vendor ID y Product ID de Pinguino) y le envía los datos. Al ser un dispositivo HID, no requiere que instales controladores (drivers) adicionales en tu sistema operativo, lo cual elimina uno de los mayores puntos de fricción en el desarrollo con microcontroladores. El proceso es transparente: el IDE compila el código, genera un archivo hexadecimal temporal y lo inyecta directamente en la memoria Flash del chip.

Versiones y Portabilidad

Existen diferentes versiones del IDE, siendo la Pinguino X una de las más estables y utilizadas. Una característica muy valorada por la comunidad es su portabilidad; es posible tener el IDE en una memoria USB con todos sus compiladores y bibliotecas configurados, y ejecutarlo en cualquier computadora sin necesidad de permisos de administrador para instalar software.

Diferencias Clave con Arduino IDE

Aunque estéticamente son parecidos, el IDE de Pinguino ofrece un control más directo sobre la configuración del hardware. Por ejemplo, permite configurar los Configuration Bits del PIC (como el Watchdog Timer, el tipo de oscilador o la protección de código) de una manera más granular. Además, mientras que el ecosistema Arduino está muy centrado en C++, Pinguino mantiene una estructura que se siente muy cercana al C estándar, lo cual es preferido por muchos ingenieros que buscan un control más estricto del uso de la memoria.

1. Arquitectura y Encapsulado del Microcontrolador (PIC32MX220F032D-I/PT)

La elección de este modelo específico en formato TQFP-44 (Thin Quad Flat Package) ofrece ventajas críticas sobre los tradicionales encapsulados DIP (Through-Hole) o chips de 8 bits:

- **Densidad de Integración:** El encapsulado TQFP reduce drásticamente el área (footprint) necesaria en la PCB, permitiendo un diseño final más compacto. Aunque es tecnología de montaje superficial (SMD), el paso (pitch) de sus pines aún permite la soldadura manual o el uso de stencils caseros sin requerir equipo industrial complejo.
- **Peripheral Pin Select (PPS):** La familia PIC32MX220 incluye mapeo de pines reconfigurables. Esto es una ventaja masiva al momento de realizar el ruteo (routing) de las pistas en el software de diseño, ya que permite reasignar funciones como UART, SPI o PWM a los pines físicos que resulten más convenientes para evitar cruces de pistas o el uso excesivo de vías.
- **Hardware USB Integrado:** Al contar con un transceptor USB 2.0 Full-Speed on-chip, se elimina la necesidad de circuitos integrados puente (como el CH340 o FT232). Esto reduce la lista de materiales (BOM), disminuye el consumo de energía y ahorra espacio en la placa.

2. Transición a Componentes Pasivos SMD (Formato 0603)

La lista de materiales especifica resistencias y capacitores en tamaño 0603. Justificar el uso de SMD frente a Through-Hole (THT) es clave en el diseño moderno:

- **Integridad de Señal:** A las frecuencias de operación de un sistema de 32 bits (hasta 40-50 MHz en el bus del sistema y 12 Mbps en el bus USB), los componentes de orificio pasante introducen inductancia y capacitancia parásita debido a sus largos terminales metálicos. Los componentes SMD 0603 minimizan estos efectos, mejorando la estabilidad de las señales de alta velocidad.
- **Manufacturabilidad:** El tamaño 0603 es el "punto dulce" para el prototipado avanzado. Es lo suficientemente pequeño para lograr una alta densidad de componentes en la PCB, pero lo suficientemente grande para ser manipulado con pinzas de precisión bajo un microscopio o lupa de banco, facilitando el ensamblaje o reparación de las primeras iteraciones de la placa.

3. Topología de Gestión de Energía (Conector Li-Po, Jack 5V y USB)

El diseño contempla múltiples fuentes de alimentación (Batería 3.7V DW02R, Jack 5V YDJ-1136 y USB). Esta redundancia y versatilidad debe justificarse orientada a la aplicación final:

- Autonomía para Aplicaciones de Control y Sensores: Integrar un conector JST directo para baterías Li-Po de 3.7V transforma la tarjeta de una simple placa de escritorio a un nodo apto para sistemas autónomos, recolección de datos remotos o sistemas de control portátiles.
- Aislamiento y Regulación: Al manejar entradas de voltaje variables (los picos de un motor de DC externo, por ejemplo, frente a la alimentación limpia del USB), un diseño que separe la lógica de carga y regulación permite proteger el núcleo del microcontrolador (que opera a 3.3V y 1.8V internamente).

4. Conectividad Física (Conector USB Mini-B SMD)

Aunque el estándar actual se inclina hacia el USB-C, el uso de conectores Mini-B o Micro-B SMD en prototipos tiene una justificación robusta:

- Esfuerzo Mecánico: Los conectores USB son el punto de mayor estrés mecánico en una tarjeta de desarrollo. Utilizar una variante SMD que incluye "pads de anclaje" (pestañas metálicas laterales que se sueldan a grandes planos de tierra en la PCB) garantiza que las fuerzas de inserción y extracción del cable no arranquen las delicadas pistas de datos (D+ y D-) de la placa.

Sugerencias para el ruteo de la placa (PCB Design)

Para documentar el proceso de diseño físico, puedes añadir que se implementarán las siguientes reglas de diseño (Design Rules):

1. Par Diferencial para USB: Las pistas del D+ y D- se trazarán juntas, en paralelo, y con la misma longitud exacta para garantizar la impedancia característica de 90 ohmios requerida por el estándar USB.
2. Planos de Tierra (GND Pours): Se utilizarán polígonos de cobre sólido en las capas superior e inferior para proporcionar un camino de retorno de baja impedancia, disipar el calor térmico del regulador de voltaje y proteger (apantallar) al microcontrolador del ruido electromagnético de alta frecuencia.
3. Desacoplo Estratégico: Los capacitores cerámicos se posicionarán a menos de 5 milímetros de los pines de alimentación (VDD/VSS y VCAP) del PIC32 para asegurar respuestas transitorias instantáneas.

Diseño propuesto

